
Introducción a Node.js

JavaScript en el servidor

Índice de Contenidos

1. ¿Qué es Node.js?
2. Características principales
 - Modelo de E/S sin bloqueo
3. Casos de uso de Node.js
 - Ideal para
 - No recomendado para
4. Instalación de Node.js
 - Para Windows, macOS y Linux
 - Verificar la instalación
5. Mi primer programa con Node.js
 - Hola Mundo
 - Ejemplo más completo: Servidor web básico
6. Módulos en Node.js
 - Tipos de módulos
 - Crear y usar un módulo local
7. Módulos core importantes
 - fs (File System)
 - path
 - http/https

8. npm: Node Package Manager
 - Iniciar un proyecto Node.js
 - Instalar un paquete
 - package.json
9. El modelo de eventos de Node.js
 - EventEmitter
 - Ejemplo práctico: servidor con eventos
10. Programación asíncrona en Node.js
 - Callbacks
 - Promesas
 - Async/Await
11. Frameworks y librerías populares
 - Web Frameworks
 - ORM y bases de datos
 - Otras herramientas útiles
12. Buenas prácticas en Node.js
13. Recursos para seguir aprendiendo
 - Documentación oficial
 - Tutoriales y cursos

¿Qué es Node.js?

Node.js es un entorno de ejecución para JavaScript construido con el motor V8 de Chrome. Node.js permite ejecutar código JavaScript en el servidor, fuera del navegador. Fue creado por Ryan Dahl en 2009 y ha revolucionado la forma en que se desarrollan aplicaciones web.

Definición oficial: "Node.js es un entorno de ejecución para JavaScript orientado a eventos asíncronos, diseñado para construir aplicaciones en red escalables."

A diferencia de la mayoría de los lenguajes que operan de forma sincrónica (bloqueante), Node.js utiliza un modelo de operaciones E/S sin bloqueo y orientado a eventos, lo que lo hace ligero y eficiente, ideal para aplicaciones en tiempo real con datos intensivos que se ejecutan en dispositivos distribuidos.

Características principales

Asíncrono y orientado a eventos

Las APIs de Node.js son no bloqueantes, lo que significa que no esperan a que una operación termine para continuar con el siguiente código.

Rápido

Gracias al motor V8 de Google Chrome, la ejecución de código en Node.js es muy rápida.

Single thread pero altamente escalable

Utiliza un solo hilo con bucle de eventos para manejar múltiples conexiones concurrentes.

Modelo de E/S sin bloqueo

Diferencia entre programación síncrona (bloqueante) y asíncrona (no bloqueante):

Ejemplo Síncrono (bloqueante):

- `// Pseudocódigo`
- `leerArchivo('archivo.txt');`
- `// Espera a que termine la lectura`

- `console.log('Archivo leído');`

```
procesarDatos(); // Se ejecuta después de leer el archivo
```

Ejemplo Asíncrono (no bloqueante en Node.js):

- `// JavaScript en Node.js`
- `fs.readFile('archivo.txt', (err, data) => {`
- `if (err) throw err;`
- `console.log('Archivo leído');`
- `});`

```
procesarDatos(); // Se ejecuta inmediatamente sin esperar a que termine la lectura
```

En el ejemplo asíncrono, Node.js no espera a que termine la lectura del archivo para continuar con la ejecución del código. En su lugar, continúa ejecutando `procesarDatos()` y cuando termina la lectura del archivo, ejecuta la función callback.

Casos de uso de Node.js

Ideal para:

- **Aplicaciones en tiempo real:** chats, juegos online, colaboración en tiempo real
- **API RESTful:** servicios web y microservicios
- **Aplicaciones de streaming:** audio, video
- **Aplicaciones de una sola página (SPA):** con backends en Node.js
- **Herramientas de línea de comandos**
- **Aplicaciones IoT (Internet de las cosas)**

No recomendado para:

- Aplicaciones con uso intensivo de CPU (procesamiento pesado)
- Aplicaciones monolíticas tradicionales

Instalación de Node.js

Para Windows, macOS y Linux:

1. Visitar el sitio oficial: <https://nodejs.org>
2. Descargar la versión LTS (recomendada para la mayoría de los usuarios)
3. Seguir el asistente de instalación

Verificar la instalación:

Abre una terminal o línea de comandos y ejecuta:

```
node -v v20.11.0 npm -v 10.2.4
```

Estos comandos mostrarán las versiones de Node.js y npm (Node Package Manager) instaladas.

npm (Node Package Manager) es el gestor de paquetes de Node.js que permite instalar y administrar dependencias/bibliotecas de terceros.

Mi primer programa con Node.js

Hola Mundo

1. Crea un archivo llamado `app.js`
2. Escribe el siguiente código:

```
console.log('¡Hola Mundo desde Node.js!');
```

3. Ejecuta el programa desde la terminal:

```
node app.js ;Hola Mundo desde Node.js!
```

Verás el mensaje "¡Hola Mundo desde Node.js!" en la consola.

Ejemplo más completo: Servidor web básico

Crea un archivo `servidor.js` con el siguiente contenido:

```
● // Importar el módulo HTTP de Node.js
● const http = require('http');
●
● // Definir el puerto donde escuchará el servidor
● const PUERTO = 3000;
●
● // Crear el servidor
● const servidor = http.createServer((req, res) => {
●   // Establecer el encabezado de respuesta con tipo de contenido
  texto plano
●   res.writeHead(200, {'Content-Type': 'text/plain'});
●
●   // Enviar el cuerpo de la respuesta
●   res.end('¡Mi primer servidor con Node.js!\n');
● });
●
● // Iniciar el servidor
● servidor.listen(PUERTO, () => {
●   console.log(`Servidor ejecutándose en
  http://localhost:${PUERTO}/`);
● });
```

Ejecuta el servidor:

```
node servidor.js Servidor ejecutándose en http://localhost:3000/
```

Ahora abre tu navegador y visita <http://localhost:3000>. Deberías ver el mensaje "¡Mi primer servidor con Node.js!".

Módulos en Node.js

Node.js utiliza el sistema de módulos CommonJS para organizar y reutilizar código. Los módulos permiten encapsular funcionalidades relacionadas y exponerlas de manera controlada.

Tipos de módulos:

1. **Módulos core:** incorporados en Node.js (como http, fs, path)
2. **Módulos locales:** creados por el desarrollador
3. **Módulos de terceros:** instalados vía npm

Crear y usar un módulo local:

Archivo `matematica.js` (nuestro módulo):

```
● // Funciones que exportaremos
● const sumar = (a, b) => a + b;
● const restar = (a, b) => a - b;
● const multiplicar = (a, b) => a * b;
● const dividir = (a, b) => a / b;
●
● // Exportamos las funciones para que estén disponibles
● module.exports = {
●   sumar,
```

- `restar,`
- `multiplicar,`
- `dividir`

```
};
```

Archivo `app.js` (uso del módulo):

- `// Importamos nuestro módulo`
- `const matematica = require('./matematica');`
-
- `// Usamos las funciones de nuestro módulo`
- `console.log(matematica.sumar(5, 3)); // 8`
- `console.log(matematica.restar(10, 4)); // 6`
- `console.log(matematica.multiplicar(3, 4)); // 12`

```
console.log(matematica.dividir(20, 5)); // 4
```

Módulos core importantes

fs (File System)

Permite trabajar con el sistema de archivos:

- `const fs = require('fs');`
-
- `// Leer un archivo de forma asíncrona`
- `fs.readFile('archivo.txt', 'utf8', (err, data) => {`
- `if (err) {`
- `console.error('Error al leer el archivo:', err);`
- `return;`
- `}`
- `console.log('Contenido del archivo:', data);`
- `});`
-

- `// Escribir en un archivo`
- `fs.writeFile('nuevo.txt', 'Contenido del archivo', (err) => {`
- `if (err) {`
- `console.error('Error al escribir el archivo:', err);`
- `return;`
- `}`
- `console.log('Archivo creado correctamente!');`

`});`

path

Proporciona utilidades para trabajar con rutas de archivos y directorios:

- `const path = require('path');`
-
- `// Unir segmentos de ruta`
- `const rutaCompleta = path.join(__dirname, 'carpeta', 'archivo.txt');`
- `console.log(rutaCompleta);`
-
- `// Obtener la extensión de un archivo`
- `const extension = path.extname('archivo.txt');`
- `console.log(extension); // .txt`
-
- `// Obtener el nombre del archivo sin extensión`
- `const nombreArchivo = path.basename('archivo.txt', '.txt');`

`console.log(nombreArchivo); // archivo`

http/https

Para crear servidores HTTP y realizar peticiones:

- `const https = require('https');`
-
- `// Realizar una petición HTTP GET`

```
● https.get('https://jsonplaceholder.typicode.com/posts/1', (res) => {  
●   let data = '';  
●  
●   // Evento cuando recibimos datos  
●   res.on('data', (chunk) => {  
●     data += chunk;  
●   });  
●  
●   // Evento cuando termina la respuesta  
●   res.on('end', () => {  
●     console.log(JSON.parse(data));  
●   });  
●   }).on('error', (err) => {  
●     console.error('Error en la petición:', err.message);  
●  
●   });
```

npm: Node Package Manager

npm es el gestor de paquetes de Node.js que facilita la instalación, actualización y gestión de dependencias.

Iniciar un proyecto Node.js:

```
npm init
```

Este comando crea un archivo `package.json` que contiene la información del proyecto y sus dependencias.

package.json

Ejemplo de un archivo package.json básico:

- {
- "name": "mi-aplicacion",
- "version": "1.0.0",
- "description": "Mi primera aplicación con Node.js",
- "main": "app.js",
- "scripts": {
- "start": "node app.js",
- "dev": "nodemon app.js",
- "test": "echo \"Error: no test specified\" && exit 1"
- },
- "author": "Tu Nombre",
- "license": "MIT",
- "dependencies": {
- "express": "^4.18.2"
- },
- "devDependencies": {
- "nodemon": "^2.0.22"
- }

}

El modelo de eventos de Node.js

Node.js utiliza un modelo basado en eventos donde ciertos objetos (emisores) emiten eventos que provocan la ejecución de funciones (listeners).

EventEmitter

El módulo `events` proporciona la clase `EventEmitter`, que es fundamental para manejar eventos en Node.js:

- `const EventEmitter = require('events');`
-
- `// Crear un emisor de eventos`
- `const emisor = new EventEmitter();`
-

- `// Registrar un listener (escuchador) para el evento 'mensaje'`
- `emisor.on('mensaje', (texto) => {`
- `console.log('Mensaje recibido:', texto);`
- `});`
-
- `// Emitir el evento 'mensaje'`
- `emisor.emit('mensaje', '¡Hola mundo de eventos!');`

Ejemplo práctico: servidor con eventos

- `const http = require('http');`
- `const server = http.createServer();`
-
- `// El servidor es un EventEmitter`
- `server.on('request', (req, res) => {`
- `res.writeHead(200, {'Content-Type': 'text/plain'});`
- `res.end('Hola desde el servidor!\n');`
- `});`
-
- `server.on('listening', () => {`
- `console.log('Servidor escuchando en el puerto 3000');`
- `});`
-
- `server.on('error', (err) => {`
- `console.error('Error en el servidor:', err.message);`
- `});`
-
- `// Iniciar el servidor en el puerto 3000`

```
server.listen(3000);
```

Programación asíncrona en Node.js

Callbacks

La forma tradicional de manejar operaciones asíncronas:

```
• fs.readFile('archivo.txt', 'utf8', (err, data) => {  
•   if (err) {  
•     console.error('Error:', err);  
•     return;  
•   }  
•   console.log('Datos:', data);  
• })
```

Promesas

Una alternativa moderna para manejar operaciones asíncronas:

```
• // Usando el módulo fs/promises  
• const fs = require('fs/promises');  
•  
• fs.readFile('archivo.txt', 'utf8')  
•   .then(data => {  
•     console.log('Datos:', data);  
•   })  
•   .catch(err => {  
•     console.error('Error:', err);  
•   })  
• })
```

Async/Await

La forma más moderna y legible de trabajar con operaciones asíncronas:

```
• const fs = require('fs/promises');  
•
```

- `async function leerArchivo() {`
- `try {`
- `const data = await fs.readFile('archivo.txt', 'utf8');`
- `console.log('Datos:', data);`
- `} catch (err) {`
- `console.error('Error:', err);`
- `}`
- `}`
-
-

```
leerArchivo();
```

Importante: Async/await es sintaxis de ES2017 (ES8) y está disponible en Node.js desde la versión 7.6.0. Es la forma recomendada de manejar operaciones asíncronas en Node.js moderno.

Frameworks y librerías populares

Web Frameworks

- **Express.js:** El framework web más popular y minimalista
- **Nest.js:** Framework inspirado en Angular para crear aplicaciones escalables
- **Koa.js:** Creado por el mismo equipo de Express, más ligero y moderno
- **Fastify:** Enfocado en proporcionar la mejor experiencia de desarrollador con el menor overhead

ORM y bases de datos

- **Sequelize:** ORM para bases de datos SQL
- **Mongoose:** ODM para MongoDB
- **Prisma:** ORM de próxima generación

Otras herramientas útiles

- **Socket.io:** Biblioteca para aplicaciones en tiempo real
- **Nodemon:** Utilidad que monitorea cambios y reinicia la aplicación
- **PM2:** Gestor de procesos de producción
- **Jest/Mocha:** Frameworks de pruebas

Buenas prácticas en Node.js

- **Usar el manejo de errores adecuado:** siempre captura y maneja errores en operaciones asíncronas
- **Estructurar bien tu proyecto:** organiza tu código en módulos y carpetas lógicas
- **Usar variables de entorno:** para configuraciones sensibles como claves API, contraseñas, etc.
- **Implementar logging:** usa bibliotecas como Winston o Morgan
- **Escribir pruebas:** unitarias, de integración y de extremo a extremo
- **Validar entrada del usuario:** nunca confíes en los datos de entrada
- **Evitar el callback hell:** usa promesas o async/await
-